lab06

April 21, 2026

0.1 Fast Fourier Transform

[31]:
```
from numpy import *
from numpy.fft import fft, ifft, fftfreq, fftshift
from matplotlib.pyplot import *

def ditfft2(x):
    N = len(x)
    if N == 1:
        return x
    else:
        even = ditfft2(x[::2])
        odd = exp(-2*pi*1j/N*arange(N/2))*ditfft2(x[1::2])
        return hstack([even+odd, even - odd])
```

[33]:
```
x = arange(16)
fft(x)
```

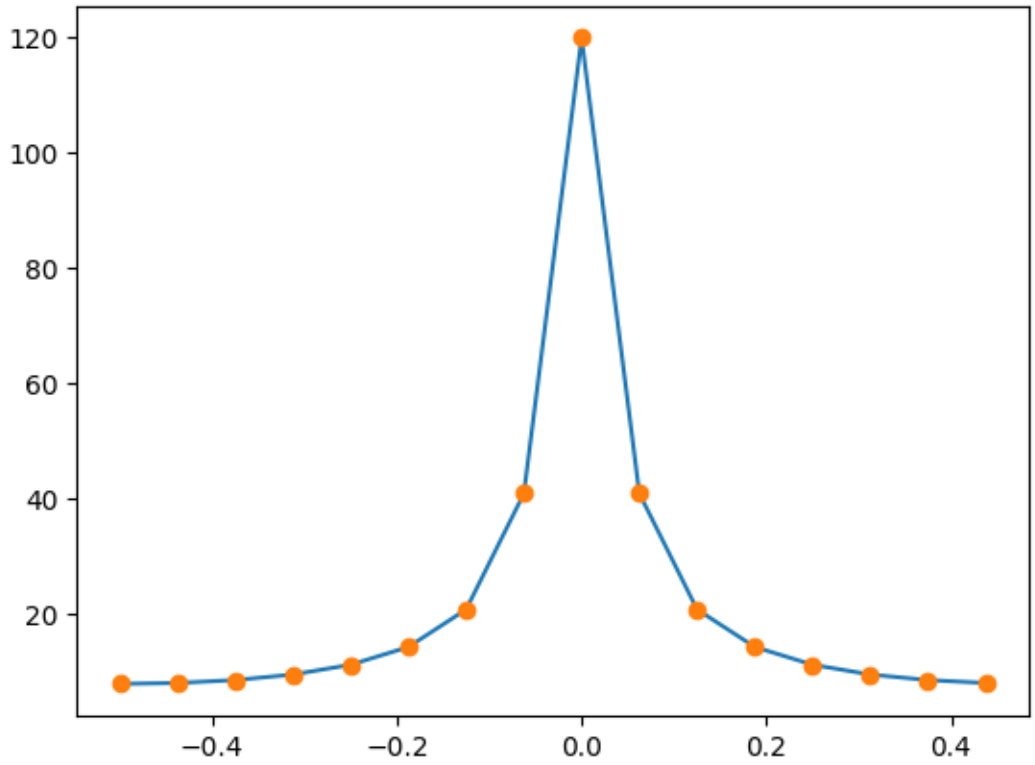
[33]:
```
array([120. +0.j        ,  -8.+40.21871594j,  -8.+19.3137085j ,
       -8.+11.9728461j ,  -8. +8.j        ,  -8. +5.3454291j ,
       -8. +3.3137085j ,  -8. +1.59129894j,  -8. +0.j        ,
       -8. -1.59129894j,  -8. -3.3137085j ,  -8. -5.3454291j ,
       -8. -8.j        ,  -8.-11.9728461j ,  -8.-19.3137085j ,
       -8.-40.21871594j])
```

[34]:
```
ditfft2(x)
```

[34]:
```
array([120. +0.j        ,  -8.+40.21871594j,  -8.+19.3137085j ,
       -8.+11.9728461j ,  -8. +8.j        ,  -8. +5.3454291j ,
       -8. +3.3137085j ,  -8. +1.59129894j,  -8. +0.j        ,
       -8. -1.59129894j,  -8. -3.3137085j ,  -8. -5.3454291j ,
       -8. -8.j        ,  -8.-11.9728461j ,  -8.-19.3137085j ,
       -8.-40.21871594j])
```

[35]:
```
plot(fftshift(fftfreq(len(x))), fftshift(abs(    fft(x))), '.-')
plot(fftshift(fftfreq(len(x))), fftshift(abs(ditfft2(x))), 'o' )
```

[35]:
```
[<matplotlib.lines.Line2D at 0x739e4cfa66f0>]
```



[111]:
```
# complex constants
2+0.5j
```

[111]: (2+0.5j)

0.2 Zero-padding

Zero-padding does **not** improve the true resolution

$$\Delta f = \frac{f_s}{N},$$

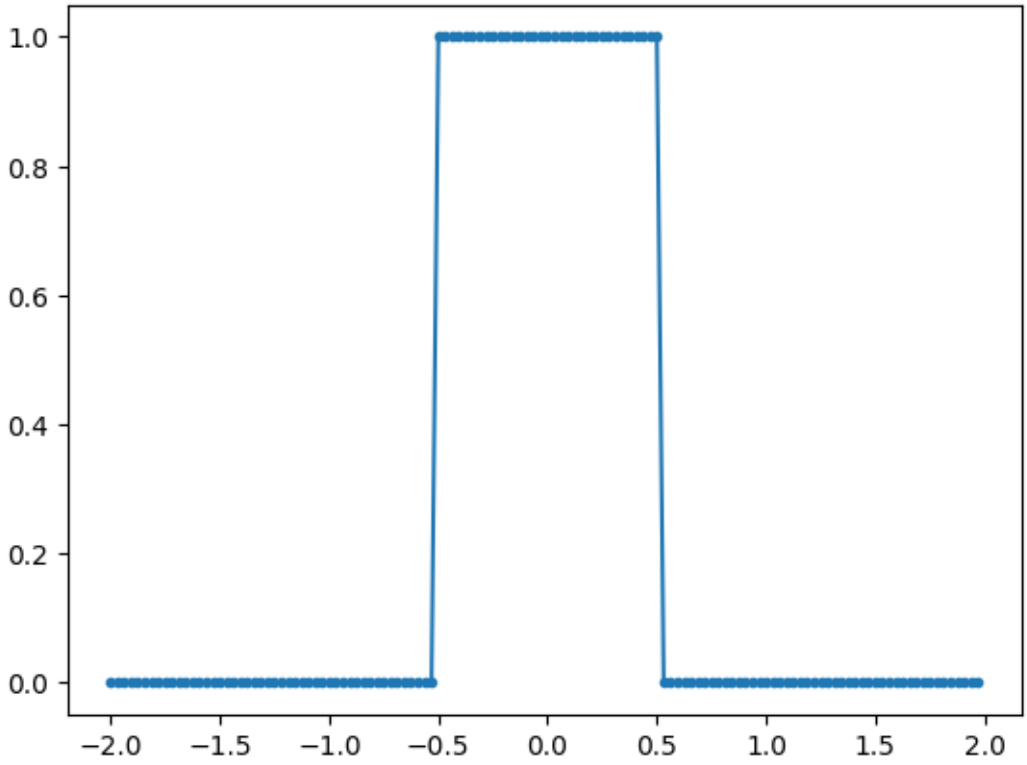
but it gives a denser plotted spectrum, so peaks are easier to read.

[36]:
```
N = 128
tmax = 2
T = 2*tmax/N
tm = 0.5
```

[37]:
```
t = arange(-tmax, tmax, T)
x = zeros(N)
#x[abs(t) <= tm] = (1 - abs(t)/tm)[abs(t) <= tm]
x[abs(t) <= tm] = 1
```

[38]:
```
plot(t, x, '.-')
```

[38]: [<matplotlib.lines.Line2D at 0x739e4cfb3c20>]

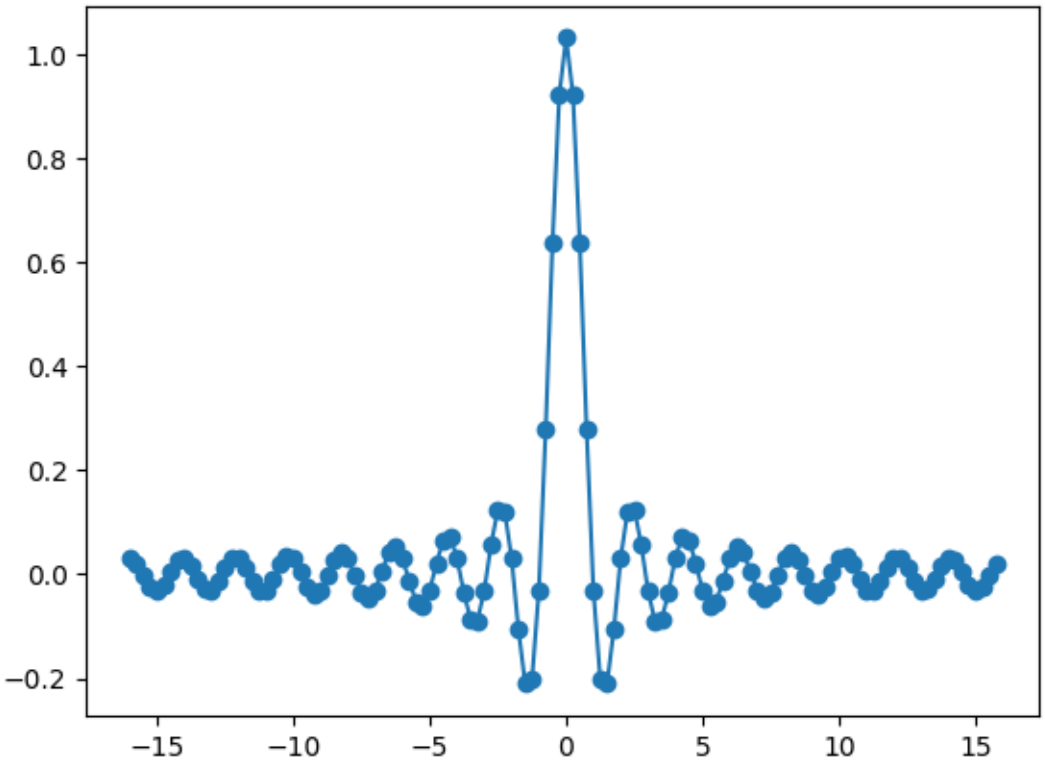


[39]:
```
X = T*fft(fftshift(x))
```

[40]:
```
f = fftfreq(N, T)
```

[56]:
```
plot(fftshift(f), fftshift(real(X)), 'o-')
#xlim([-16, 16])
```

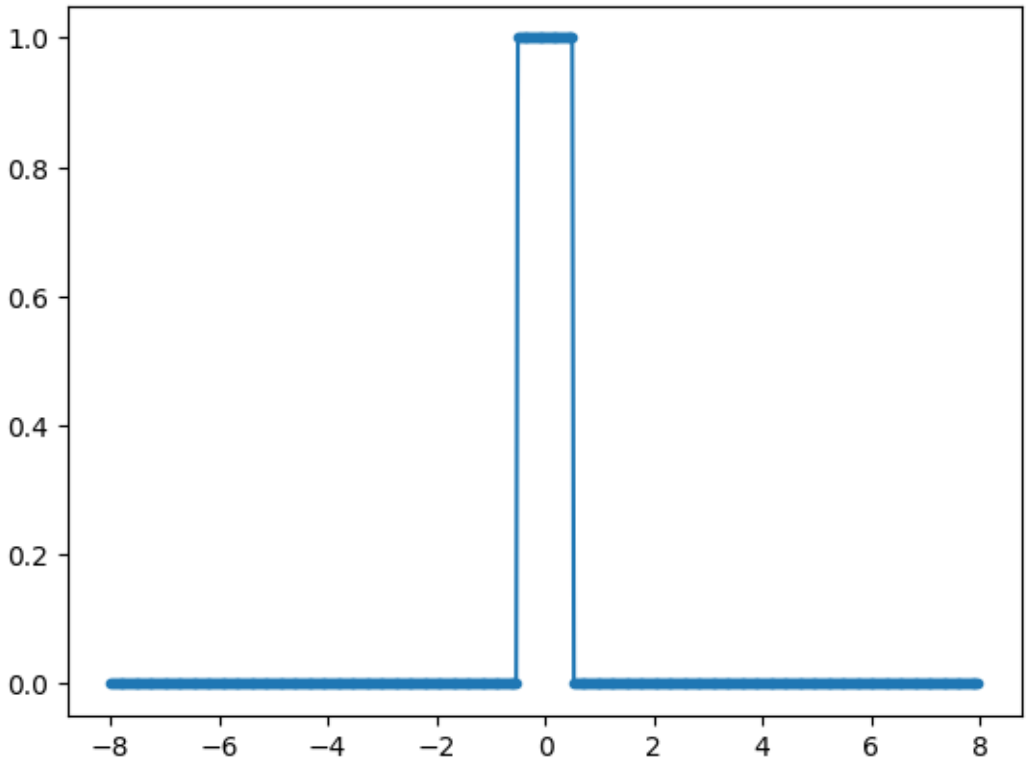
[56]: [<matplotlib.lines.Line2D at 0x739e47d2e7e0>]



[58]:
```
Npad = 512
xp = pad(x,((Npad - N)//2, (Npad - N)//2))
tpmax = tmax*Npad//N
tp = arange(-tpmax, tpmax, T)
```

[51]:
```
plot(tp, xp, '.-')
```

[51]:
```
[<matplotlib.lines.Line2D at 0x739e47cc04d0>]
```

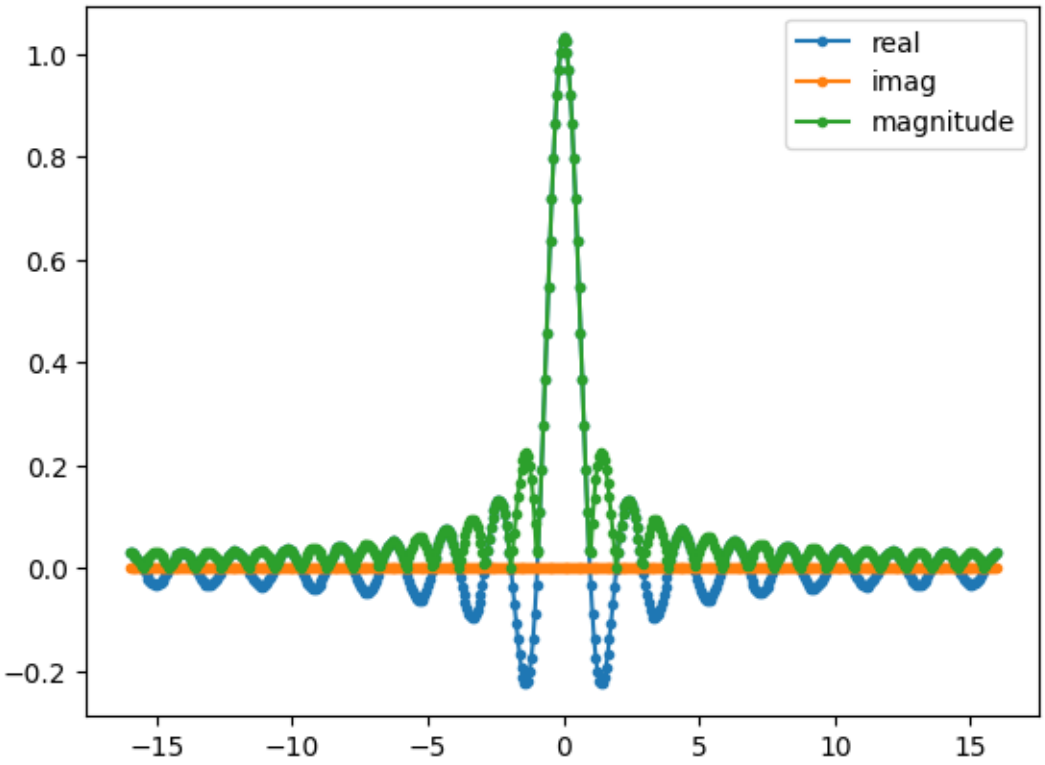


[19]:
```
__version__
```

[19]: '1.26.4'

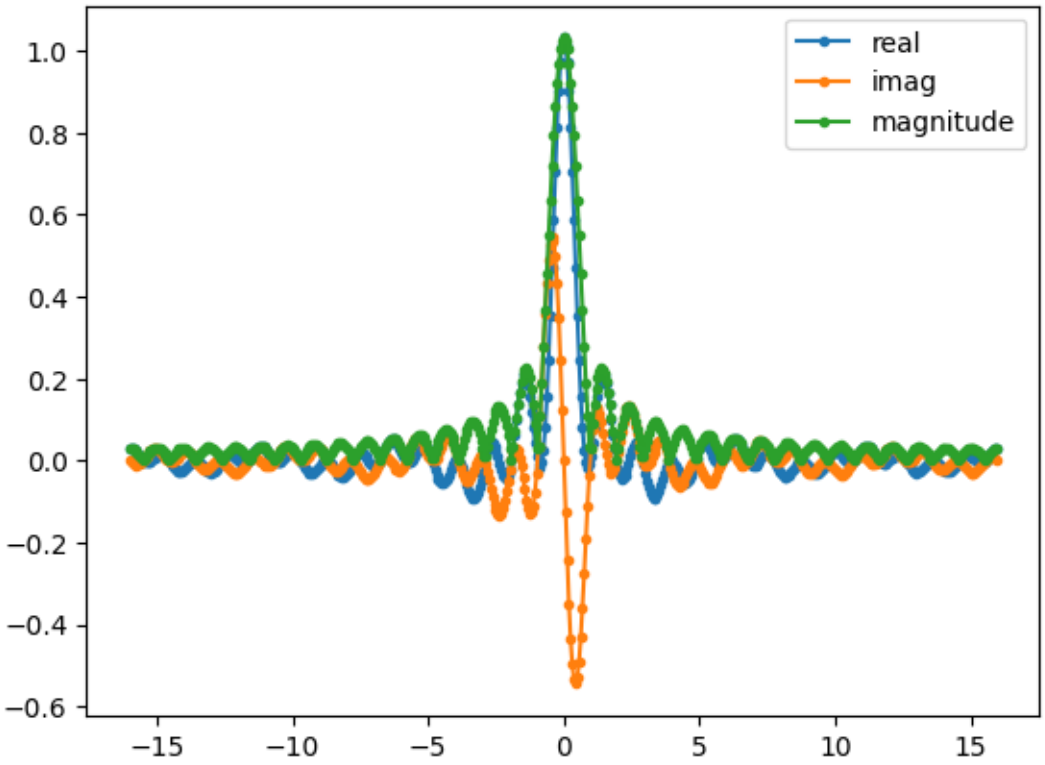
[75]:
```
fp = fftfreq(Npad, T)
Xp = T*fft(roll(fftshift(xp), 0))
plot(fftshift(fp), (fftshift((real(Xp)))), '.-', label='real')
plot(fftshift(fp), (fftshift((imag(Xp)))), '.-', label='imag')
plot(fftshift(fp), (fftshift((abs(Xp)))), '.-', label='magnitude')
#xlim([-3.5, 3.5])
legend()
```

[75]: <matplotlib.legend.Legend at 0x739e479088c0>



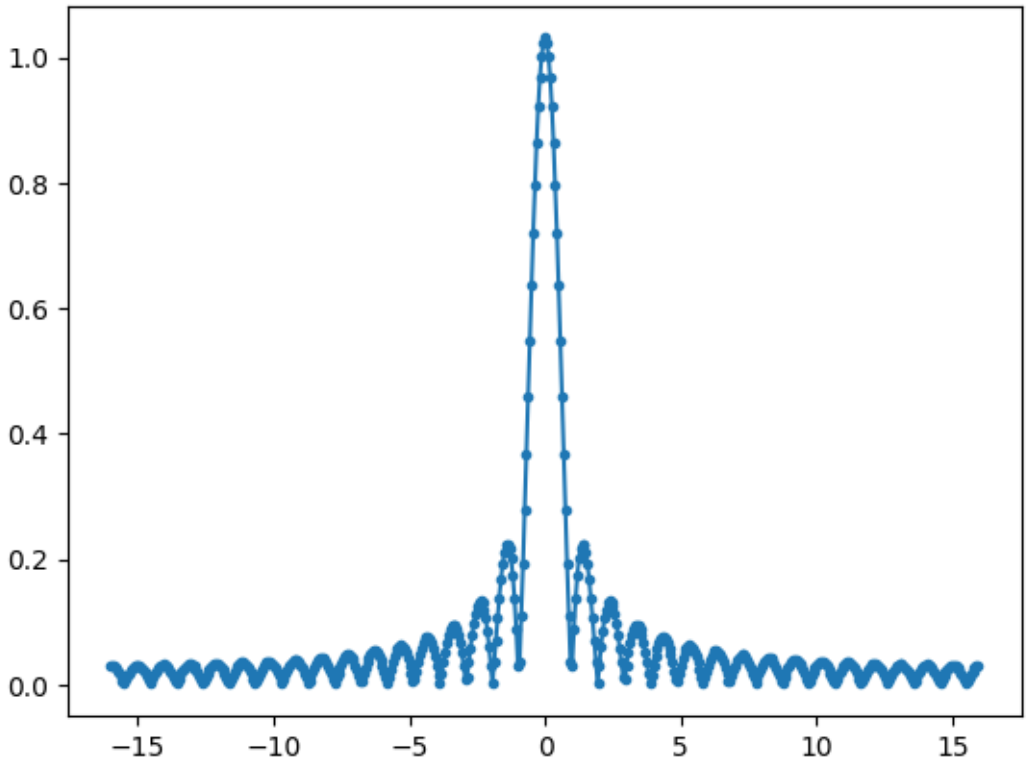
[76]:
```
# again: shifting introduces extra phase factor (oscillations)
# but the magnitude is not affected
Xp = T*fft(roll(fftshift(xp), 10)) # shifting
plot(fftshift(fp), (fftshift((real(Xp)))), '.-', label='real')
plot(fftshift(fp), (fftshift((imag(Xp)))), '.-', label='imag')
plot(fftshift(fp), (fftshift((abs(Xp)))), '.-', label='magnitude')
#xlim([-3.5, 3.5])
legend()
```

[76]:
```
<matplotlib.legend.Legend at 0x739e479a4fb0>
```



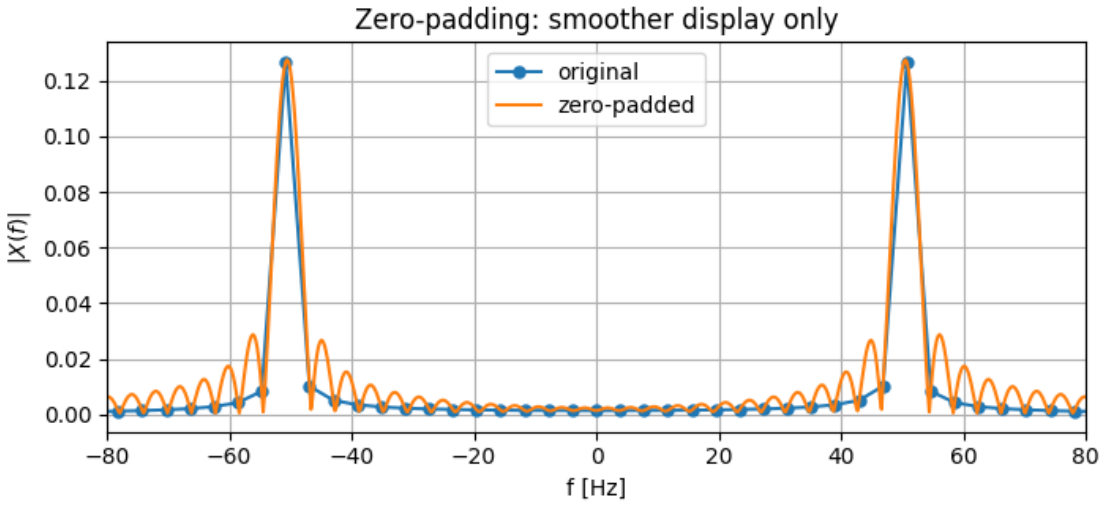
[79]:
```
# different (simpler) padding if you don't care about oscillations due to phase␣
↪shifts (look at the magnitude)
xp = pad(x, ( Npad -N, 0))
Xp = T*fft(xp)
plot(fftshift(fp), abs(fftshift((Xp))), '.-')
#xlim([-1.5, 1.5])
```

[79]: [<matplotlib.lines.Line2D at 0x739e47815ac0>]



[142]:
```
fs = 1000.0
T = 1 / fs
N = 256
t = np.arange(N) / fs
x = np.cos(2 * np.pi * 50.5 * t)

# Original FFT
X1 = T * np.fft.fft(x)
f1 = np.fft.fftfreq(N, T)

# Zero-padded FFT
Npad = 4096
xpad = np.pad(x, (0, Npad - N))
X2 = T * np.fft.fft(xpad)
f2 = np.fft.fftfreq(Npad, T)

plt.figure(figsize=(8, 3.2))
plt.plot(np.fft.fftshift(f1), np.abs(np.fft.fftshift(X1)), '.-',␣
 ↪label="original", markersize=10)
plt.plot(np.fft.fftshift(f2), np.abs(np.fft.fftshift(X2)), label="zero-padded")
plt.xlim(-80, 80)
plt.xlabel("f [Hz]")
```

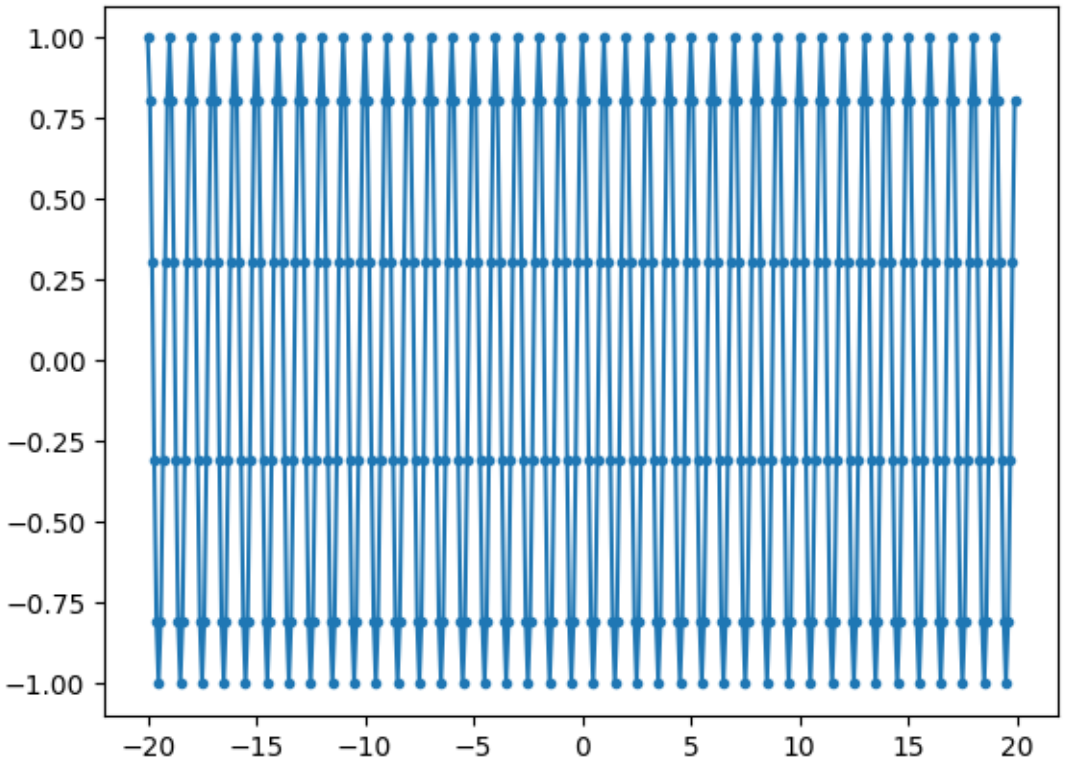
```
plt.ylabel(r"$|X(f)|$")
plt.title("Zero-padding: smoother display only")
plt.grid(True)
plt.legend()
plt.show()
```



Leakage

[117]:
```
T = 0.1
tau = 10*T
tmax = 20*tau
t = arange(-tmax, tmax, T)
x = cos(2*pi*t/tau)
plot(t, x, '.-')
#xlim([-5, 5])
```

[117]: [<matplotlib.lines.Line2D at 0x739e472313a0>]

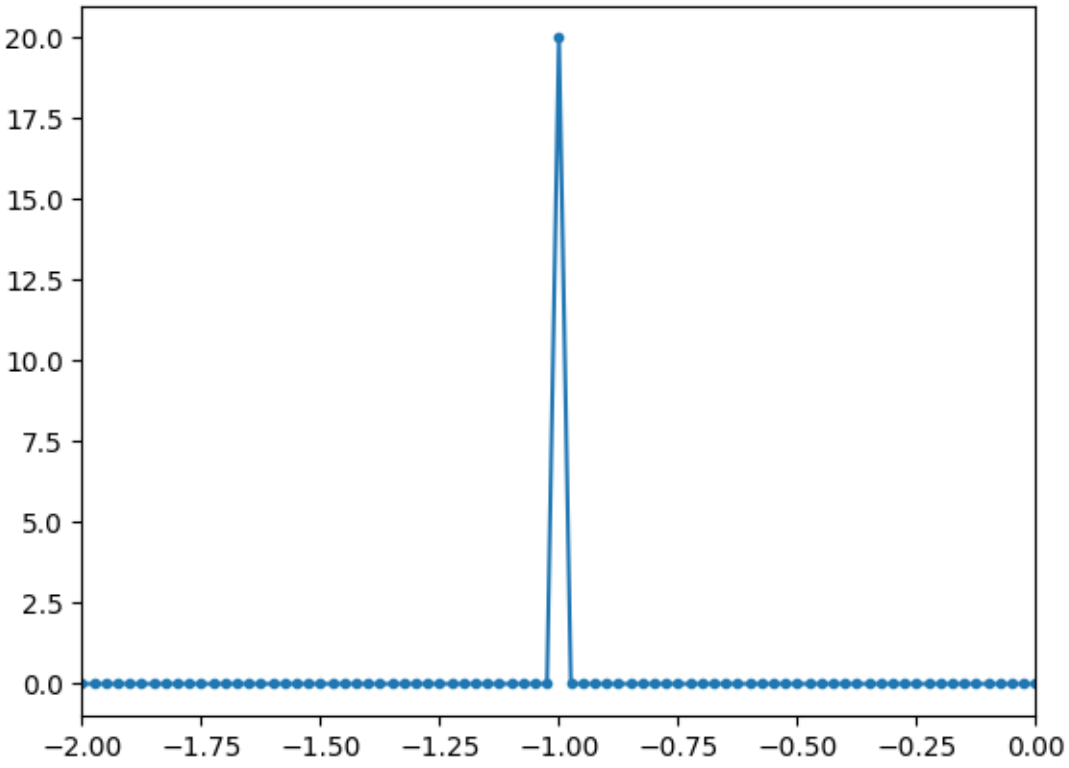


[121]:
```
X = T*fft(fftshift(x))  # should be a Dirac delta
```

[122]:
```
f = fftfreq(len(X), T)
```

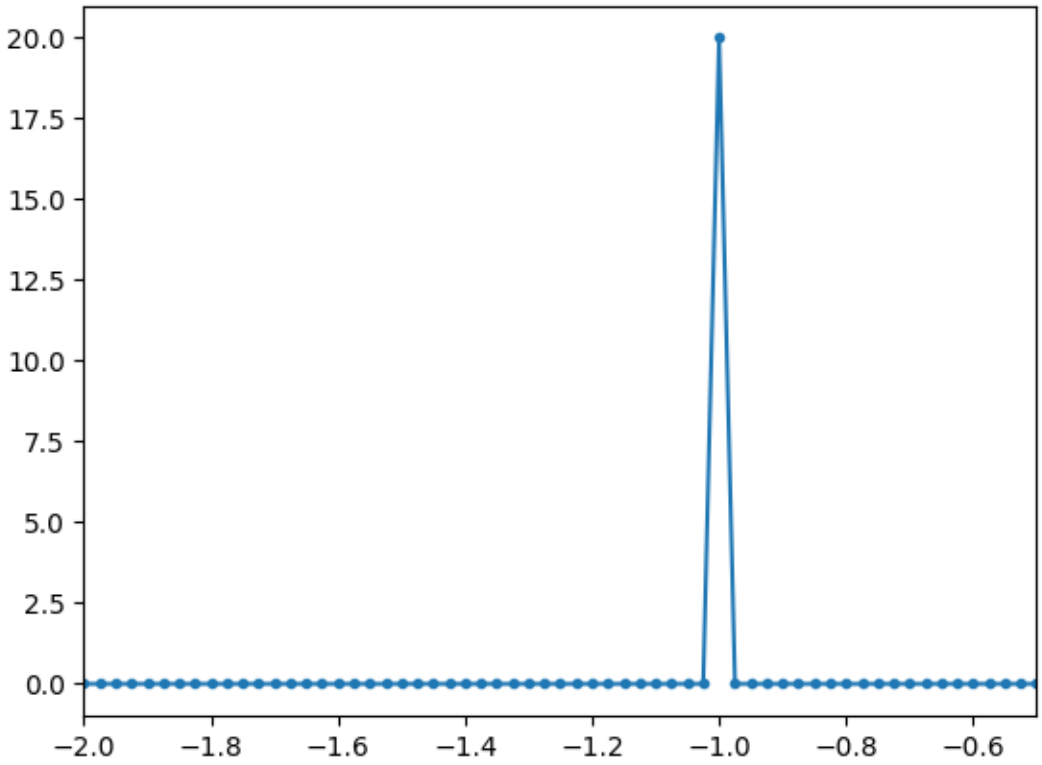
[123]:
```
plot(fftshift(f), fftshift(real(X)), '.-')
xlim([-2, 0])
```

[123]: (-2.0, 0.0)



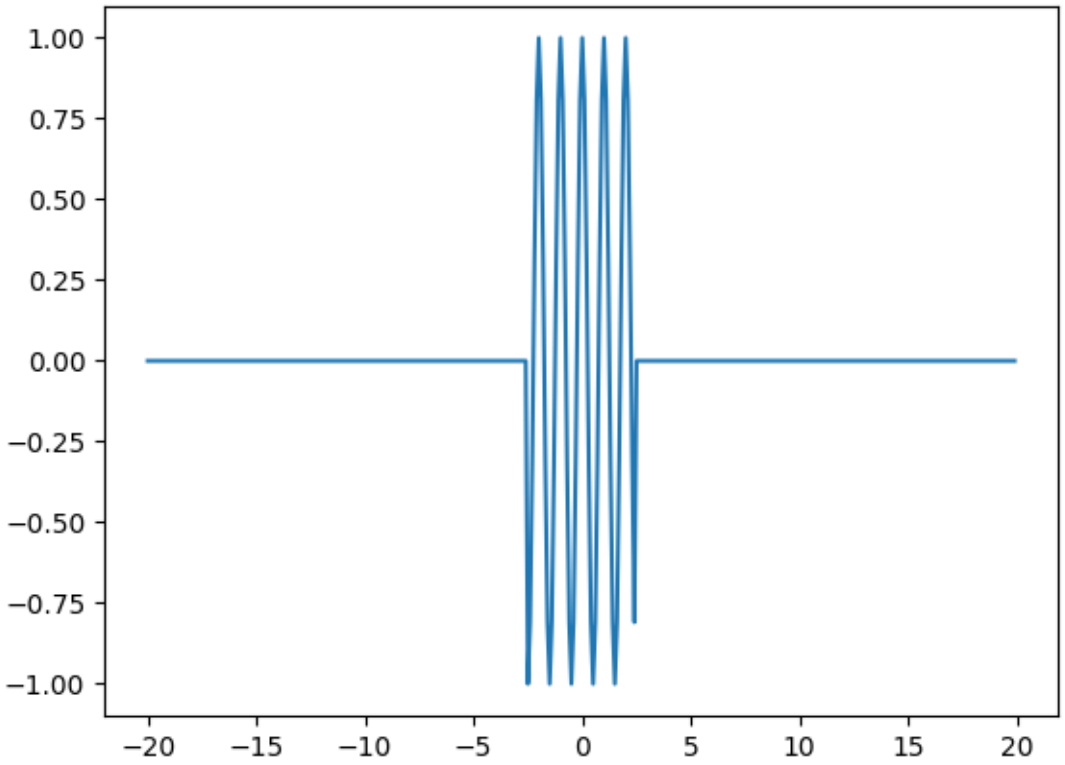
[124]:
```
X = T*fft(fftshift(x))
f = fftfreq(len(X), T)
plot(fftshift(f), fftshift(abs(X)), '.-')
xlim([-2, -0.5])
```

[124]: (-2.0, -0.5)



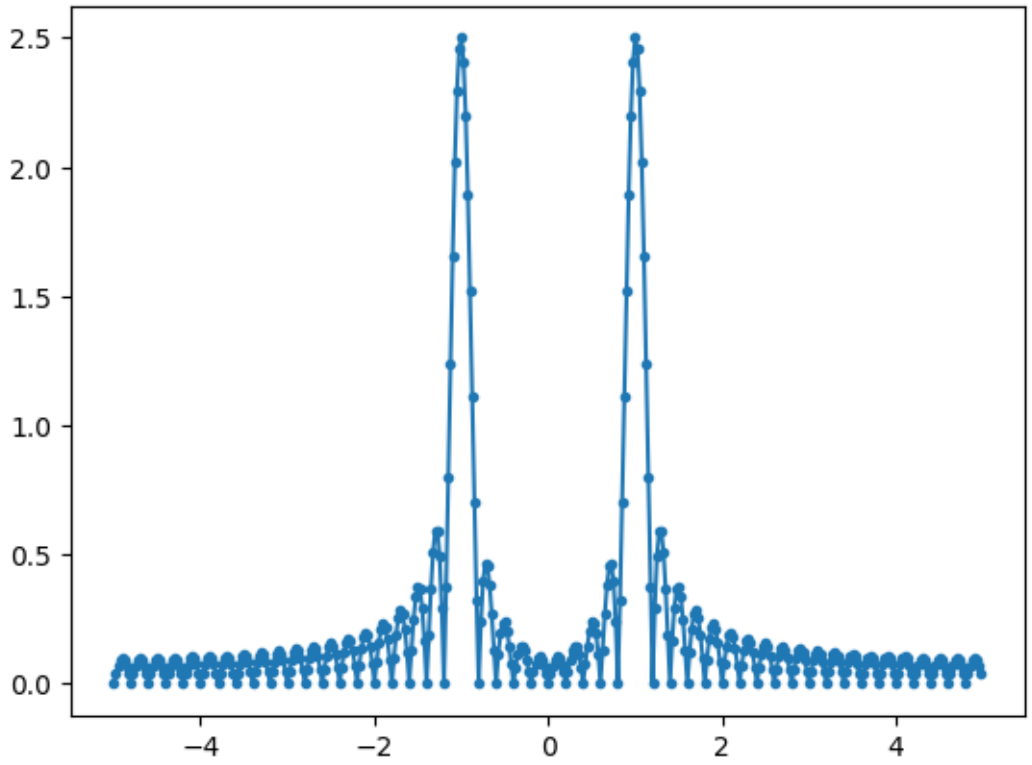
[125]:
```
window = 2.5
x[abs(t) > window] = 0
plot(t, x)
```

[125]: [<matplotlib.lines.Line2D at 0x739e47127530>]



[126]:
```
X = T*fft(fftshift(x))
plot(fftshift(f), fftshift(abs(X)), '.-')
#xlim([-2, -0.5])
```

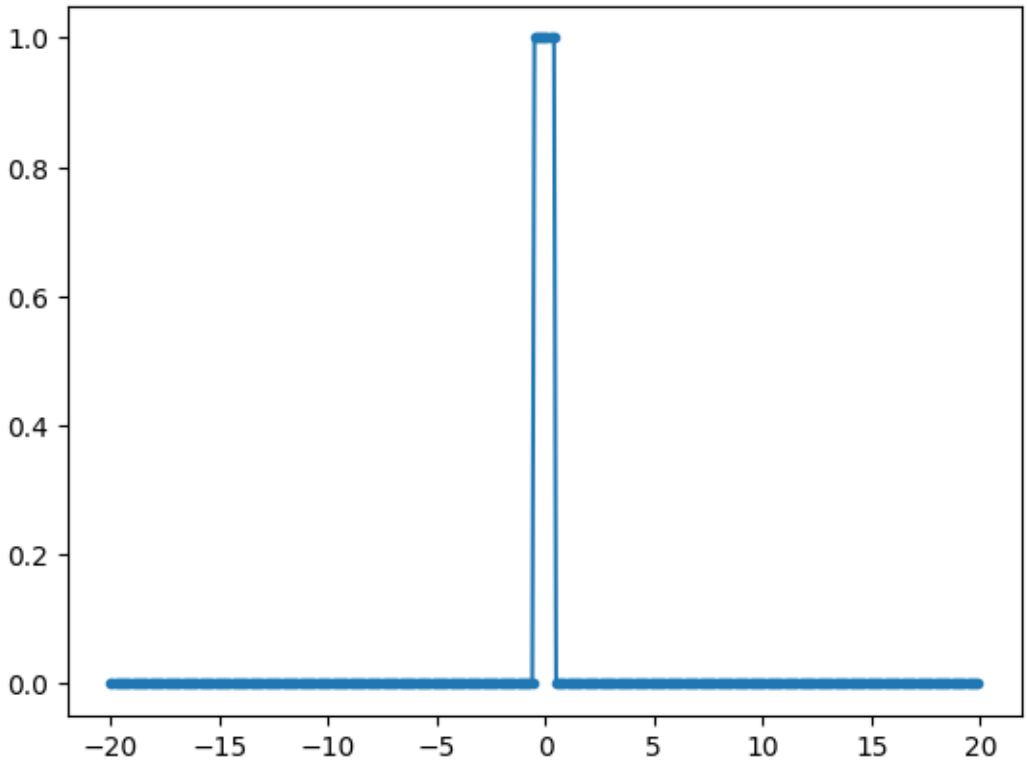
[126]: [<matplotlib.lines.Line2D at 0x739e47d55c40>]



[127]:
```
def rect(t):
    return (abs(t) <= 1/2).astype(float)
```

[128]:
```
plot(t, rect(t), '.-')
```

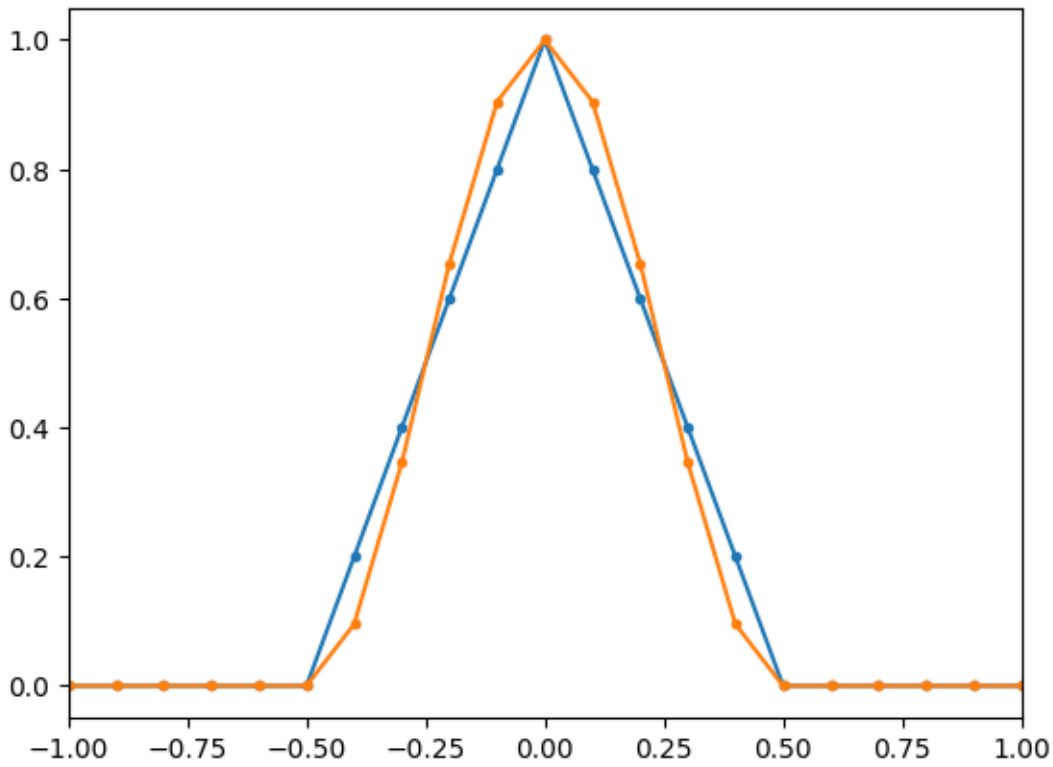
[128]:
```
[<matplotlib.lines.Line2D at 0x739e4719b230>]
```



[129]:
```
def triangle(t):
    res = zeros(len(t))
    res[abs(t) < 0.5] = (1 - 2*abs(t))[abs(t) < 0.5]
    return res
def Hann(t):
    res = zeros(len(t))
    res[abs(t) < 0.5] = (cos(pi*abs(t))**2)[abs(t) < 0.5]
    return res
```

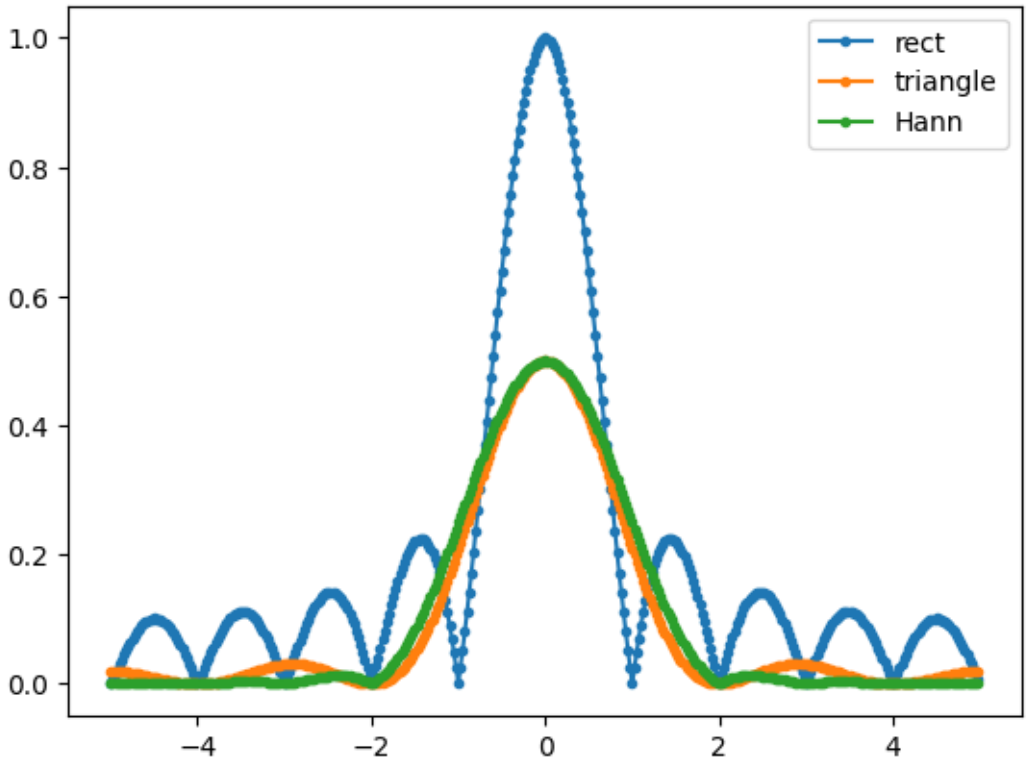
[130]:
```
plot(t, triangle(t), '.-')
plot(t, Hann(t), '.-')
xlim([-1, 1])
```

[130]:
```
(-1.0, 1.0)
```



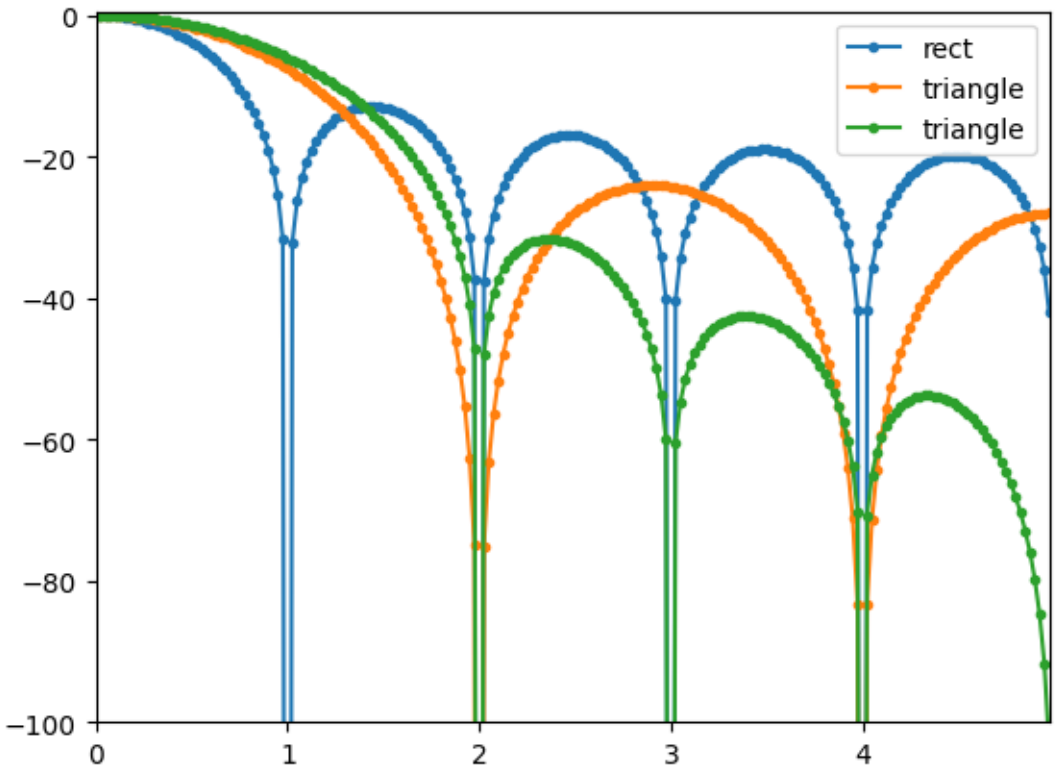
[131]:
```
Xrect = T*fft((rect    (t)))
Xtri  = T*fft((triangle(t)))
XHann = T*fft((Hann(t)))
plot(fftshift(f), fftshift(abs(Xrect)), '.-', label='rect')
plot(fftshift(f), fftshift(abs(Xtri )), '.-', label='triangle')
plot(fftshift(f), fftshift(abs(XHann)), '.-', label='Hann')
legend()
```

[131]:
```
<matplotlib.legend.Legend at 0x739e47d54fb0>
```



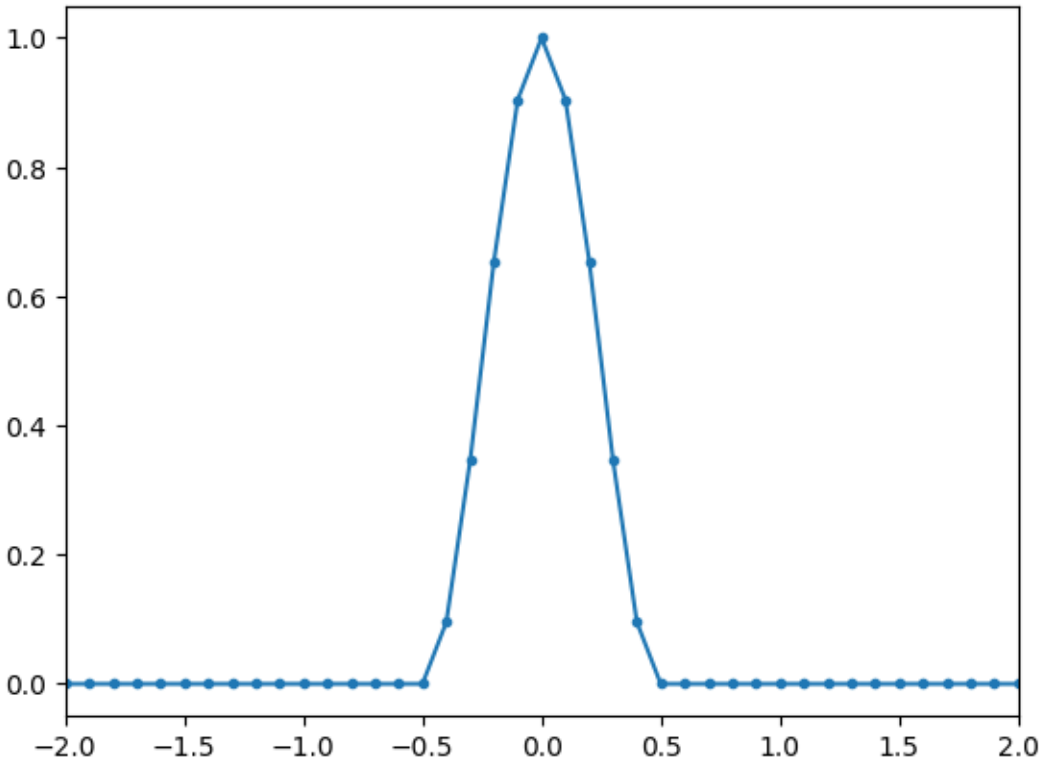
[132]:
```
plot(fftshift(f), 20*log10(fftshift(abs(Xrect)+1e-20)/abs(Xrect).max()), '.-',␣
↪label='rect')
plot(fftshift(f), 20*log10(fftshift(abs(Xtri )+1e-20)/abs(Xtri ).max()), '.-',␣
↪label='triangle')
plot(fftshift(f), 20*log10(fftshift(abs(XHann)+1e-20)/abs(XHann).max()), '.-',␣
↪label='triangle')
xlim([0, f.max()])
ylim([-100, 0.5])
legend()
```

[132]: `<matplotlib.legend.Legend at 0x739e4f3588c0>`



[133]:
```
plot(t, Hann(t), '.-')
xlim([-2, 2])
```

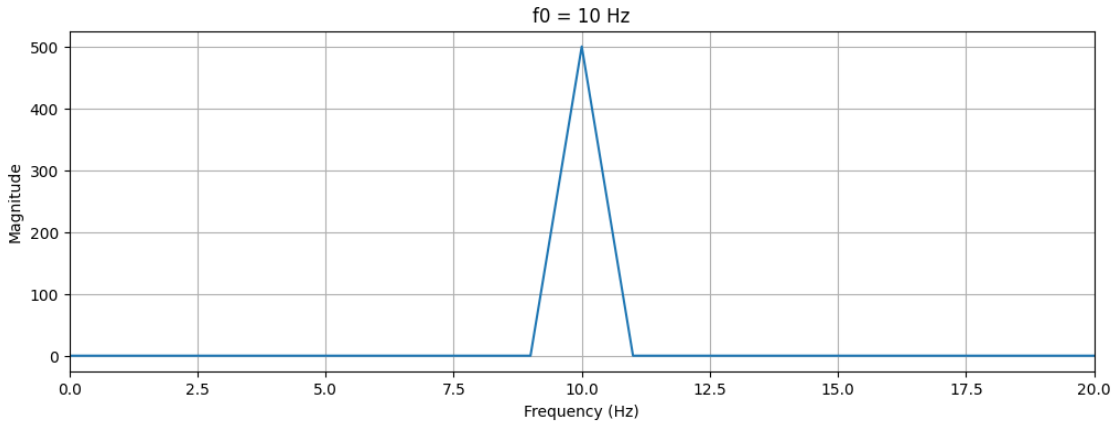
[133]: (-2.0, 2.0)

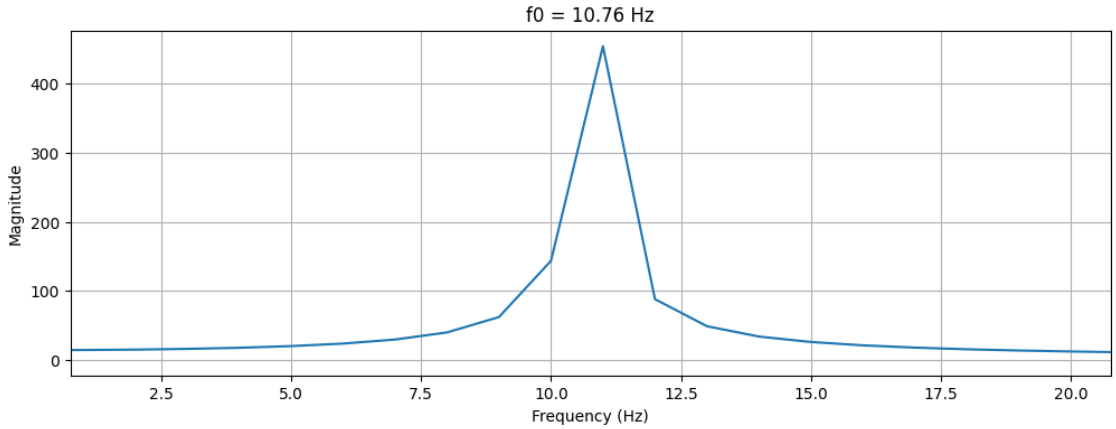


0.3 Scalloping

[134]:
```python
import numpy as np
import matplotlib.pyplot as plt

fs = 1000
N = 1000
n = np.arange(N)

def show_fft(f0):
    x = np.cos(2*np.pi*f0*n/fs)
    X = np.fft.fft(x)
    f = np.fft.fftfreq(N, 1/fs)

    plt.figure(figsize=(12,4))
    plt.plot(f[:N//2], np.abs(X[:N//2]))
    plt.title(f"f0 = {f0} Hz")
    plt.xlabel("Frequency (Hz)")
    plt.ylabel("Magnitude")
    plt.xlim([f0 -10, f0+10])
    plt.grid()
    plt.show()
```

```
# Perfect alignment
show_fft(10)

# Off-bin → scalloping loss
show_fft(10.76)
```

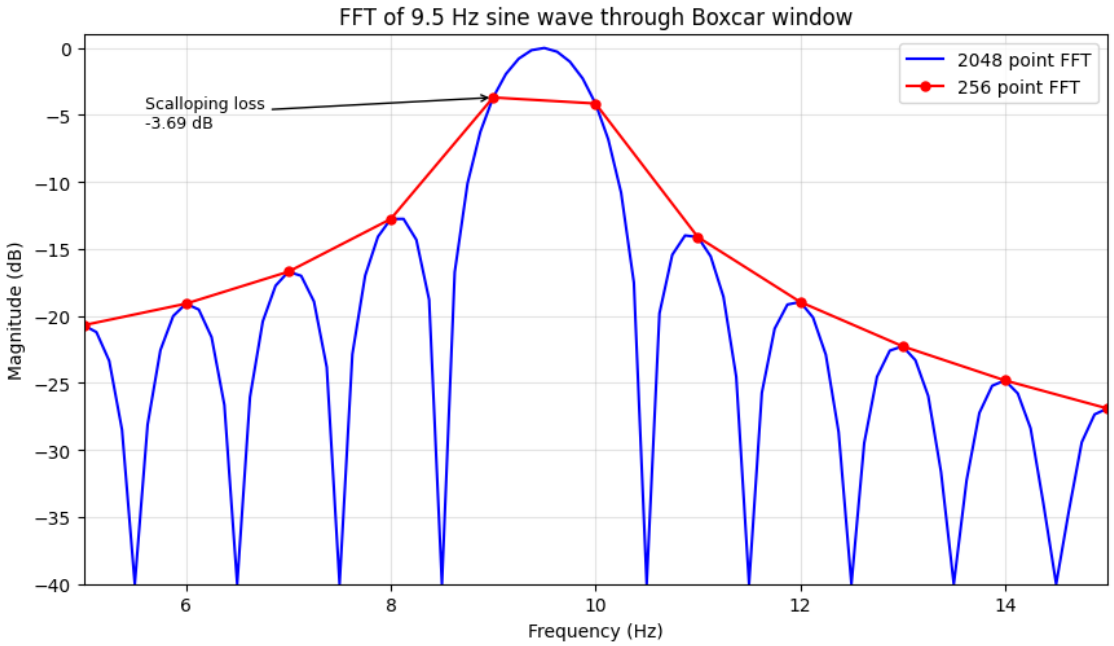




[ ]:
```
T = 0.15
f0 = 1
period = 1/f0#10*T
#tmax = 5*period
t = arange(-tmax, tmax, T)
x = cos(2*pi*t/period)
plot(t, x, '.-')
```

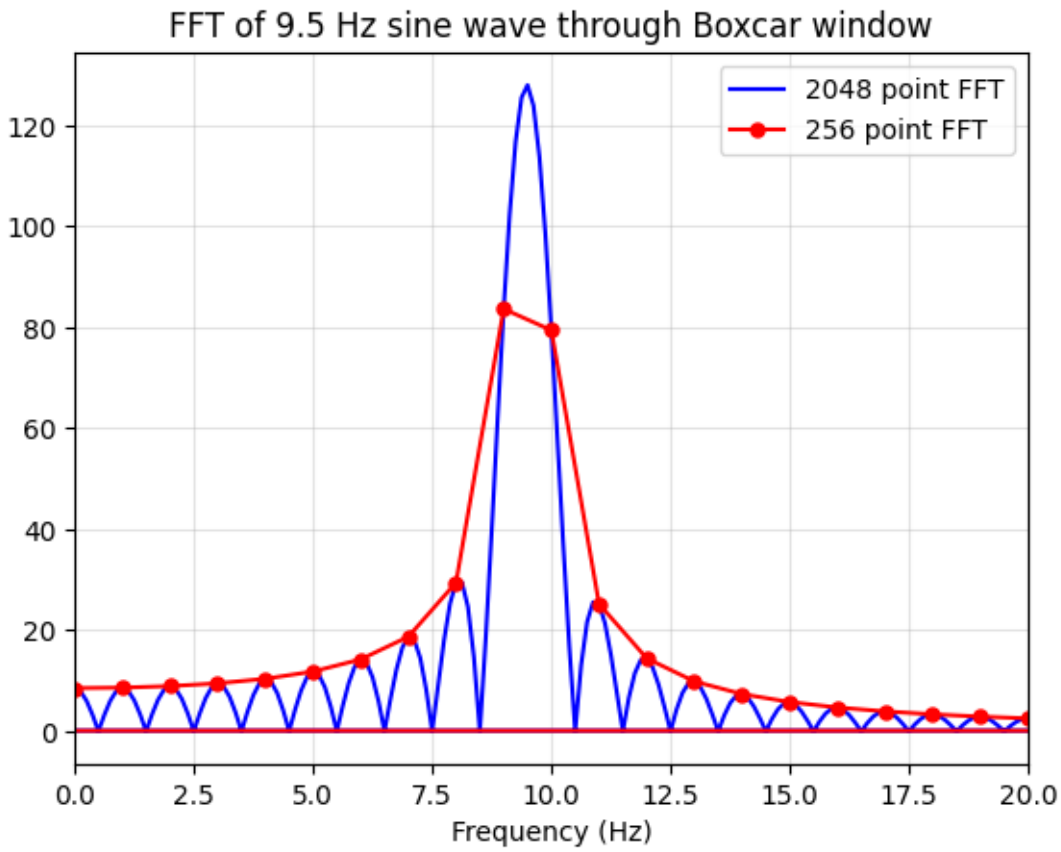
[141]:
```
import numpy as np
import matplotlib.pyplot as plt

# Parameters
fs = 256          # Hz
N = 256           # original record length
Npad = 2048       # zero-padded FFT length
f0 = 9.5          # Hz, exactly halfway between 9 and 10 Hz bins for 256-point␣
 ↪FFT

# Time-domain signal
n = np.arange(N)
x = np.sin(2 * np.pi * f0 * n / fs)

# FFTs
X_256 = np.fft.rfft(x, n=N)
X_2048 = np.fft.rfft(x, n=Npad)

# Frequency axes
f_256 = np.fft.rfftfreq(N, d=1/fs)
f_2048 = np.fft.rfftfreq(Npad, d=1/fs)

# Normalize both to the peak of the zero-padded spectrum
ref = np.max(np.abs(X_2048))
mag_256_db = 20 * np.log10(np.abs(X_256) / ref)
mag_2048_db = 20 * np.log10(np.abs(X_2048) / ref)

# Avoid -inf in plotting
floor_db = -40
mag_256_db = np.maximum(mag_256_db, floor_db)
mag_2048_db = np.maximum(mag_2048_db, floor_db)

# Scalloping loss of the 256-point FFT
idx_9 = np.argmin(np.abs(f_256 - 9))
idx_10 = np.argmin(np.abs(f_256 - 10))
scallop_db = mag_256_db[idx_9]   # same as idx_10 here, by symmetry

# Plot
plt.figure(figsize=(8.8, 5.2))

plt.plot(f_2048, mag_2048_db, color='blue', lw=1.5, label='2048 point FFT')
plt.plot(f_256, mag_256_db, 'o-', color='red', ms=5, lw=1.5, label='256 point␣
 ↪FFT')

plt.xlim(5, 15)
plt.ylim(-40, 1)
plt.xlabel('Frequency (Hz)')
```

```
plt.ylabel('Magnitude (dB)')
plt.title('FFT of 9.5 Hz sine wave through Boxcar window')
plt.grid(True, alpha=0.35)
plt.legend(loc='upper right')

# Annotation
plt.annotate(
    f'Scalloping loss\n{scallop_db:.2f} dB',
    xy=(9, scallop_db),
    xytext=(5.6, -3.5),
    arrowprops=dict(arrowstyle='->', lw=1),
    fontsize=9,
    ha='left',
    va='top'
)

plt.tight_layout()
plt.show()
```



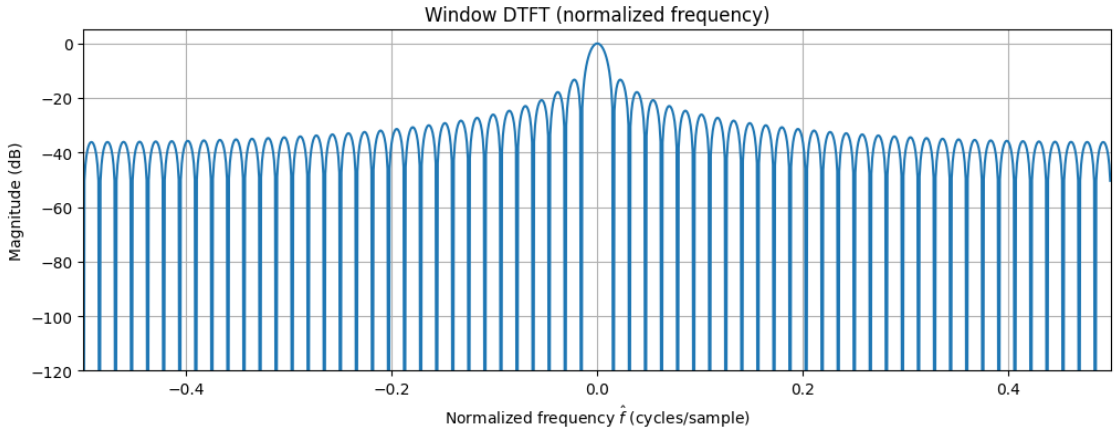
[136]:
```
import numpy as np
import matplotlib.pyplot as plt

# Parameters
fs = 256          # Hz
N = 256           # original record length
```

```
Npad = 2048        # zero-padded FFT length
f0 = 9.5           # Hz, exactly halfway between 9 and 10 Hz bins for 256-point␣
 ↪FFT

# Time-domain signal
n = np.arange(N)
x = np.sin(2 * np.pi * f0 * n / fs)

# FFTs
X_256 = np.fft.fft(x, n=N)
X_2048 = np.fft.fft(x, n=Npad)

# Frequency axes
f_256 = np.fft.fftfreq(N, d=1/fs)
f_2048 = np.fft.fftfreq(Npad, d=1/fs)


# Plot

plt.plot(f_2048, abs(X_2048), color='blue', lw=1.5, label='2048 point FFT')
plt.plot(f_256, abs(X_256), 'o-', color='red', ms=5, lw=1.5, label='256 point␣
 ↪FFT')

plt.xlim(0, 20)
# plt.ylim(-40, 1)
plt.xlabel('Frequency (Hz)')
plt.title('FFT of 9.5 Hz sine wave through Boxcar window')
plt.grid(True, alpha=0.35)
plt.legend(loc='upper right')


plt.show()
```



[137]:
```
import numpy as np
import matplotlib.pyplot as plt

# =========================
# PARAMETERS
# =========================
N = 64                      # window length
pad = 16                    # zero-padding factor (for smooth DTFT view)
Nfft = N * pad              # FFT size

# =========================
# WINDOW (change here)
# =========================
n = np.arange(N)

# Examples (uncomment one)
w = np.ones(N)                                  # Rectangular
# w = np.hanning(N)                             # Hann
# w = np.hamming(N)                             # Hamming
```

```
# w = np.blackman(N)                          # Blackman

# =========================
# ZERO-PAD (DTFT sampling)
# =========================
w_pad = np.zeros(Nfft)
w_pad[:N] = w

# =========================
# FFT → DTFT samples
# =========================
W = np.fft.fft(w_pad)
W = np.fft.fftshift(W)                        # center at 0 frequency

# =========================
# NORMALIZED FREQUENCY AXIS
# =========================
f = np.fft.fftshift(np.fft.fftfreq(Nfft, d=1))  # cycles/sample

# =========================
# MAGNITUDE (dB)
# =========================
W_mag = np.abs(W)
W_dB = 20 * np.log10(W_mag / np.max(W_mag) + 1e-12)

# =========================
# PLOTTING
# =========================
plt.figure(figsize=(12, 4))

plt.plot(f, W_dB)
plt.xlabel('Normalized frequency $\hat{f}$ (cycles/sample)')
plt.ylabel('Magnitude (dB)')
plt.title('Window DTFT (normalized frequency)')
plt.grid()

plt.ylim([-120, 5])
plt.xlim([-0.5, 0.5])

plt.show()
```



[138]:
```
x = arange(20).reshape((4, 5))
x
```

[138]:
```
array([[ 0,  1,  2,  3,  4],
       [ 5,  6,  7,  8,  9],
       [10, 11, 12, 13, 14],
       [15, 16, 17, 18, 19]])
```

[139]:
```
x.sum(axis=0)
```

[139]:
```
array([30, 34, 38, 42, 46])
```

[140]:
```
x.sum(axis=1)
```

[140]:
```
array([10, 35, 60, 85])
```

[ ]:
```

```